TÓM TẮT VÀ CẤU TRÚC BÁO CÁO

Đề tài xây dựng mô-đun nhận dạng tiếng nói tiếng Việt cho hệ thống hỏi đáp ViMMRC, đồng thời thực hành học chuyển giao và tinh chỉnh hiệu quả tham số. Mô hình nền openai/whisper-small được tinh chỉnh bằng LoRA trên hai tập con VietSpeech có tổng thời lượng khoảng 20 giờ và 50 giờ. Hai thí nghiệm tương ứng sử dụng 16 giờ và 46 giờ dữ liệu huấn luyện, cùng một tập validation khoảng 2 giờ và test khoảng 2 giờ.

Trên 1.802 mẫu test cố định, Whisper Small gốc đạt WER 36,76% và CER 19,43%. Mô hình LoRA 20 giờ đạt 15,20% / 7,29%; mô hình LoRA 50 giờ đạt 13,77% / 6,67%. So với mô hình gốc, mô hình 50 giờ giảm khoảng 62,56% WER tương đối. Trọng số tốt nhất được merge, chuyển sang CTranslate2 INT8 và tích hợp vào API FastAPI.

Kết quả phản ánh phép đo trên tập con VietSpeech, chưa phải chất lượng trên mọi giọng nói hoặc mọi câu hỏi ViMMRC. Báo cáo phân biệt kết quả đánh giá Transformers với kiểm thử chức năng của bản INT8, đồng thời chỉ rõ nguy cơ trùng miền/người nói và chồng lấn dữ liệu của mô hình tham khảo ViWhisper.

Các phần chính

Phần	Nội dung
1–2	Bài toán, vị trí STT và cơ sở Whisper–LoRA
3–4	Streaming, xử lý dữ liệu và mở rộng 20 → 50 giờ
5–7	Môi trường, huấn luyện và phương pháp đánh giá
8–10	Kết quả, đường cong validation và early stopping
11–12	Chuyển đổi mô hình và tích hợp API
13–15	Vấn đề thực tế, giới hạn và kết luận
Tài liệu / phụ lục	Nguồn tham khảo, lệnh tái lập và tệp minh chứng

Quy ước: "mô hình 20 giờ/50 giờ" là tên thí nghiệm theo tổng thời lượng dữ liệu, không có nghĩa toàn bộ 20/50 giờ được đưa vào tập train.

1. BÀI TOÁN VÀ VỊ TRÍ TRONG HỆ THỐNG

1.1. Phát biểu bài toán

Đầu vào của STT là tín hiệu âm thanh chứa câu hỏi tiếng Việt; đầu ra là chuỗi văn bản được sử dụng làm câu hỏi cho pipeline hỏi đáp. Mô hình không dự đoán đáp án ViMMRC trực tiếp. Sai sót nhận dạng, đặc biệt ở tên riêng, con số và từ khóa, có thể làm thay đổi ý nghĩa câu hỏi và ảnh hưởng đến truy hồi hoặc đọc hiểu ở các bước sau.

```
Người dùng ghi âm câu hỏi
          ↓
Angular → POST /api/stt → Whisper đã fine-tune
          ↓
Văn bản câu hỏi → Retrieval → MRC → Câu trả lời → TTS
```

Hình 1. Vai trò của STT trong luồng hỏi đáp bằng giọng nói.

1.2. Mục tiêu thực nghiệm

- Xây dựng baseline bằng Whisper Small đa ngôn ngữ, cấu hình nhận dạng tiếng Việt.
- Fine-tune mô hình nền bằng LoRA trên GPU cá nhân 16 GB, thay vì chỉ sử dụng mô hình tiếng Việt có sẵn.
- Đánh giá tác động của việc mở rộng dữ liệu train, với validation và test được giữ cố định.
- Triển khai mô hình tự huấn luyện vào ứng dụng, duy trì khả năng quay về mô hình 20 giờ.

1.3. Đóng góp và phạm vi

Đóng góp chính là một quy trình xuyên suốt từ lấy dữ liệu theo luồng, tạo manifest, tách tập, trích đặc trưng, fine-tune, đánh giá đến triển khai. Đề tài không đề xuất kiến trúc ASR mới và không huấn luyện Whisper từ đầu. Whisper đã được tiền huấn luyện đa ngôn ngữ; kết quả baseline vì vậy không được diễn giải là mô hình chưa từng học tiếng Việt.

Trong bản triển khai, STT là dịch vụ độc lập dùng giao tiếp HTTP. Việc tách dịch vụ cho phép thay mô hình nhận dạng mà không thay đổi hợp đồng dữ liệu của giao diện và các mô-đun hỏi đáp.

2. CƠ SỞ PHƯƠNG PHÁP: WHISPER VÀ LoRA

2.1. Whisper Small

Whisper là mô hình Transformer encoder–decoder cho nhận dạng tiếng nói và các tác vụ liên quan. Âm thanh được biến đổi thành đặc trưng log-Mel spectrogram, encoder tạo biểu diễn âm học, còn decoder sinh tuần tự các token văn bản. Phiên bản small đa ngôn ngữ có quy mô xấp xỉ 244 triệu tham số [1, 2].

Trong thực nghiệm, WhisperProcessor đảm nhiệm trích đặc trưng và tokenization. Âm thanh được đưa về 16 kHz mono; ngôn ngữ và tác vụ được đặt là vi và transcribe. Đây là nhận dạng cùng ngôn ngữ, không phải dịch tiếng Việt sang tiếng Anh.

2.2. Học chuyển giao và tinh chỉnh hiệu quả tham số

Full fine-tuning cập nhật toàn bộ trọng số, đòi hỏi lưu gradient và trạng thái optimizer cho rất nhiều tham số. LoRA giữ cố định trọng số nền, học một hiệu chỉnh hạng thấp trên một số lớp tuyến tính [3]. Với ma trận W_0, trọng số hiệu dụng được mô tả như sau:

$$W = W_0 + (\alpha/r)BA$$
$$W_0 \in R^{d\times k},\ B \in R^{d\times r},\ A \in R^{r\times k},\ r \ll \min(d, k)$$

Thay vì cập nhật d×k giá trị cho mỗi ma trận, LoRA học khoảng r(d+k) tham số. Trong đề tài, adapter được gắn vào q_proj và v_proj của attention, với rank 16, alpha 32 và dropout 0,05.

Lựa chọn	Vai trò
openai/whisper-small	Mô hình nền của cả hai thí nghiệm độc lập
LoRA	1.769.472 tham số trainable, khoảng 0,73% tổng tham số sau gắn adapter
BF16	Mixed precision khi train trên GPU có hỗ trợ
CTranslate2 INT8	Định dạng triển khai sau merge, không dùng để backpropagation

LoRA giảm phần tham số cần cập nhật, nhưng không loại bỏ chi phí lưu activations và không đồng nghĩa mọi thành phần của mô hình chỉ sử dụng 0,73% bộ nhớ.

3. DỮ LIỆU VÀ THÍ NGHIỆM 20 GIỜ

3.1. Nguồn dữ liệu

VietSpeech do nhóm tác giả MMLab-UIT công bố có hơn 1.100 giờ tiếng nói, 1.026.047 mẫu, tần số lấy mẫu 16 kHz và quy mô lưu trữ khoảng 131 GB theo dataset card [4]. Dữ liệu được thu thập từ nhiều nguồn xã hội với các vùng giọng khác nhau. Việc sử dụng tuân theo điều kiện truy cập của dataset; token Hugging Face chỉ được nhập trong terminal, không ghi vào mã nguồn.

3.2. Streaming và lọc mẫu

Thay vì tải toàn bộ bộ dữ liệu, script prepare_vietspeech.py dùng streaming=True, shuffle buffer 10.000 và seed 42. Dữ liệu được giải mã từ raw audio bằng soundfile rồi materialize thành FLAC. Streaming giảm nhu cầu tải trọn bộ, nhưng vẫn cần mạng, bộ đệm và dung lượng cho audio cục bộ, đặc trưng và checkpoint.

- Loại transcript rỗng; rút gọn chuỗi khoảng trắng và bỏ khoảng trắng đầu/cuối.
- Chỉ giữ audio có thời lượng trong khoảng 0,5–30 giây; bỏ mẫu không giải mã được.
- Ghi manifest JSONL chứa audio_path, transcription, duration và group.
- Cấp quota theo thời lượng từng split, nên số mẫu không bắt buộc đúng tỷ lệ 80/10/10.

Tập	Số mẫu	Thời lượng (giờ)
Train	14.393	16,0004
Validation	1.797	2,0004
Test	1.802	2,0002
Tổng	17.992	20,0010

3.3. Tách tập theo nhóm transcript

Schema thực tế không có trường người nói/nguồn thích hợp. Vì vậy group được tạo từ transcript đã chuẩn hóa khoảng trắng và chuyển chữ thường, sau đó SHA-1 ánh xạ vào các bucket train/validation/test. Các transcript trùng theo quy tắc này nằm cùng split. Cơ chế không phát hiện mọi cách viết tương đương, không khử hết audio trùng và không bảo đảm tách biệt người nói.

4. MỞ RỘNG LÊN THÍ NGHIỆM 50 GIỜ

4.1. Giữ cố định dữ liệu đánh giá

Để so sánh với thí nghiệm 20 giờ, chỉ mở rộng tập train; không chia lại validation hoặc test. Script expand_vietspeech.py kế thừa các manifest cũ, hard-link audio khi có thể và stream thêm mẫu train mới. Output 50 giờ được đặt trong thư mục riêng, tránh ghi đè dữ liệu và trọng số cũ.

Mẫu bổ sung phải không thuộc nhóm transcript của validation/test, thuộc bucket train theo cùng quy tắc hash và chưa có sample ID trong dữ liệu đã kế thừa. Việc này bảo vệ ranh giới transcript giữa các split, nhưng sample ID không phải fingerprint nội dung âm thanh nên vẫn không thay thế được kiểm tra trùng audio chuyên sâu.

Tập	20 giờ: số mẫu / giờ	50 giờ: số mẫu / giờ
Train	14.393 / 16,0004	41.212 / 46,0005
Validation	1.797 / 2,0004	1.797 / 2,0004 (giữ nguyên)
Test	1.802 / 2,0002	1.802 / 2,0002 (giữ nguyên)
Tổng	17.992 / 20,0010	44.811 / 50,0011

Số mẫu train tăng thêm 26.819, tương đương khoảng 30 giờ. Tỷ lệ thời lượng của bộ mở rộng xấp xỉ 92%/4%/4%; không còn là 80%/10%/10% vì tập đánh giá được khóa từ thí nghiệm ban đầu.

4.2. Hậu kiểm trước huấn luyện

- So sánh từng bản ghi validation/test với bộ 20 giờ, bỏ qua audio_path do thư mục đã thay đổi.
- Kiểm tra giao tập group: train–validation, train–test và validation–test đều bằng 0.
- Xác nhận đủ 44.811 audio đọc được sau khi khắc phục sự cố lưu trữ WSL.
- Giữ riêng output của run 50 giờ để mô hình 20 giờ tiếp tục là mốc so sánh và phương án dự phòng.

Kết quả mở rộng cho biết lợi ích của toàn bộ cấu hình thí nghiệm mới; không phải một ablation chỉ thay dữ liệu trong khi giữ tuyệt đối mọi yếu tố khác.

5. MÔI TRƯỜNG VÀ CẤU HÌNH HUẤN LUYỆN

Thành phần	Giá trị đã kiểm tra
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB VRAM
Hệ điều hành train	Ubuntu 24.04 trên WSL2
Python / PyTorch	3.11.17 / 2.14.1+cu130
CUDA của wheel PyTorch	13.0
Transformers / PEFT	4.57.6 / 0.21.2
Datasets / JiWER	3.6.0 / 4.0.0
CTranslate2	4.8.2

Bảng 3. Phiên bản môi trường được đối chiếu tại thời điểm lập báo cáo.

Tham số	Cấu hình
Base model	openai/whisper-small cho cả hai run
LoRA	r=16; alpha=32; dropout=0,05; q_proj, v_proj
Batch / accumulation	4 / 8 → effective batch 32 trên 1 GPU
Learning rate / warmup	1×10^{-4} / warmup ratio 0,05
Precision / epochs	BF16 mixed precision / tối đa 3 epoch
Validation / checkpoint	Mỗi 250 steps; giữ tối đa 3 checkpoint
Chọn mô hình	Validation WER thấp nhất; load_best_model_at_end
LoRA 20 giờ	1.350 steps, 3 epoch; best step 1.250
LoRA 50 giờ	2.750 steps, epoch 2,1351; best step 2.750

Run 50 giờ dùng early stopping với patience=3 và ngưỡng 0,2 điểm WER. Trong bản trainer hiện tại, cơ chế này được biểu diễn bằng EarlyStoppingCallback; lịch sử run 20 giờ cho thấy nó đã hoàn thành đủ 3 epoch. Không khẳng định mọi phiên bản lịch sử của script có cùng callback chỉ dựa trên mã hiện tại.

6. QUY TRÌNH TINH CHỈNH MÔ HÌNH

6.1. Trích đặc trưng và tạo nhãn

Audio được đọc bằng soundfile ở float32, chuyển mono khi cần và resample về 16 kHz. WhisperProcessor trích đặc trưng log-Mel và attention mask. Transcript được tokenizer mã hóa thành nhãn. Data collator padding batch, thay vị trí nhãn padding bằng −100 để không tính vào loss và xử lý token mở đầu phù hợp cho decoder.

Trích đặc trưng được thực hiện trước khi nạp mô hình train. Pipeline dùng một process preprocessing và writer batch 100 mẫu để giảm rủi ro multiprocessing/fork sau khi CUDA đã được khởi tạo và hạn chế lượng dữ liệu ghi dồn vào cache.

6.2. Tối ưu

Mô hình nền được nạp ở precision chuẩn; Seq2SeqTrainer quản lý autocast BF16 trong quá trình train. Chỉ adapter LoRA được cập nhật. Gradient accumulation gom gradient của 8 micro-batch kích thước 4 trước mỗi optimizer step. Cấu hình hiện tại không bật gradient checkpointing; không cần thêm kỹ thuật này khi run đã đáp ứng giới hạn GPU.

```
Audio → log-Mel + attention mask
Transcript → token IDs → padding được mask bằng -100
                ↓
Whisper nền cố định + LoRA trainable
                ↓
Cross-entropy → backpropagation → cập nhật adapter
                ↓
Validation generation → WER/CER → chọn best checkpoint
```

6.3. Quản lý thí nghiệm

Run 50 giờ bắt đầu lại từ openai/whisper-small thay vì tiếp tục adapter 20 giờ. Hai thư mục checkpoint, merged model và CTranslate2 được tách biệt. Sau train, best_adapter được xuất từ checkpoint có validation WER tốt nhất, không mặc định lấy epoch cuối.

Trước các run đầy đủ, pipeline đã được smoke-test trên khoảng 15 phút dữ liệu và 10 optimizer steps. Các số liệu của smoke test chỉ nhằm xác nhận đường đi kỹ thuật, không được gộp vào bảng kết quả chính hoặc dùng như kết luận khoa học.

7. PHƯƠNG PHÁP ĐÁNH GIÁ

7.1. Chỉ số WER và CER

```
WER (%) = 100 × (S + D + I) / N
CER (%) = 100 × (S_char + D_char + I_char) / N_char
```

S, D, I lần lượt là số phép thay thế, xóa và chèn khi căn chỉnh chuỗi dự đoán với nhãn chuẩn; N là độ dài chuỗi tham chiếu. CER áp dụng trên ký tự. Giá trị càng thấp càng tốt; WER không nên được diễn giải đơn giản thành "100% − WER là độ chính xác" vì có thể có lỗi chèn và thậm chí WER vượt 100%.

evaluate_models.py dùng JiWER tính metric trên toàn bộ danh sách dự đoán/tham chiếu. Script không bổ sung chuẩn hóa chữ thường, bỏ dấu câu, chuẩn hóa số hoặc tách từ tiếng Việt trước khi chấm; ngoài việc strip output và chuẩn hóa khoảng trắng ở bước tạo dữ liệu. Vì vậy chữ hoa, dấu câu, chữ số và cách phiên âm đều có thể ảnh hưởng kết quả.

> *Trong tiếng Việt, các đơn vị phân tách bằng khoảng trắng thường là âm tiết, không luôn tương ứng từ vựng hoàn chỉnh. WER ở đây là WER theo token khoảng trắng của JiWER, không phải WER sau word segmentation tiếng Việt.*

7.2. Thiết lập so sánh

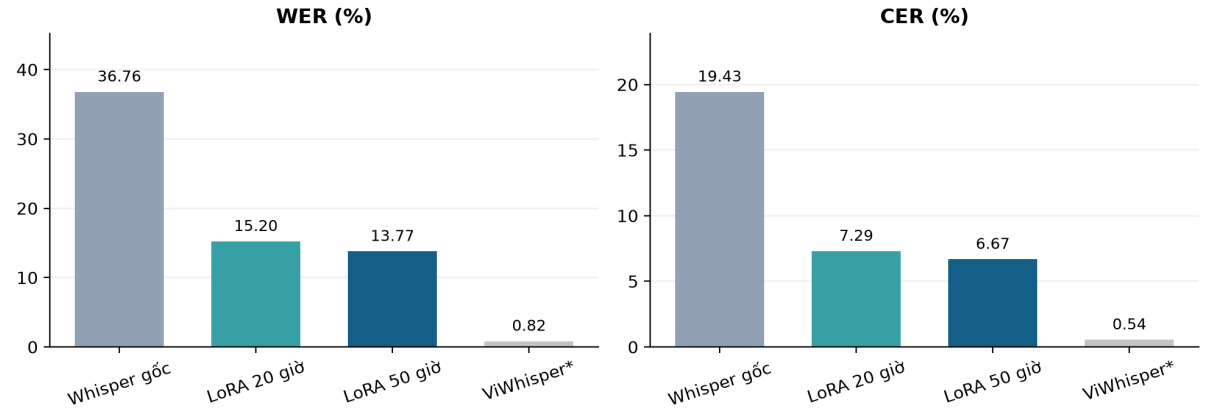
- Cùng manifest test 1.802 mẫu, khoảng 2 giờ, cho baseline và hai mô hình tự fine-tune.
- Inference Transformers trên GPU, BF16, batch 8; language=vi, task=transcribe.
- Không dùng test làm đầu vào cập nhật gradient; chọn best checkpoint bằng validation.
- ViWhisper Small là mốc tham khảo có nguy cơ chồng lấn VietSpeech, không phải đối chứng độc lập sạch.

7.3. Phân biệt đánh giá và triển khai

Các WER/CER được báo cáo là của checkpoint Transformers sau merge. Bản CTranslate2 INT8 được kiểm thử khả năng nạp, nhận dạng và API; chưa có benchmark toàn bộ test set sau lượng tử hóa. Đồng thời cấu hình giải mã production có beam_size=5 và VAD, nên không mặc nhiên suy ra metric production giống phép đo Transformers.

8. KẾT QUẢ TRÊN TEST CỐ ĐỊNH

Mô hình	WER (%)	CER (%)
openai/whisper-small	36,76	19,43
Whisper Small + LoRA, bộ 20 giờ	15,20	7,29
Whisper Small + LoRA, bộ 50 giờ	13,77	6,67
NhutP/ViWhisper-small (*)	0,82	0,54



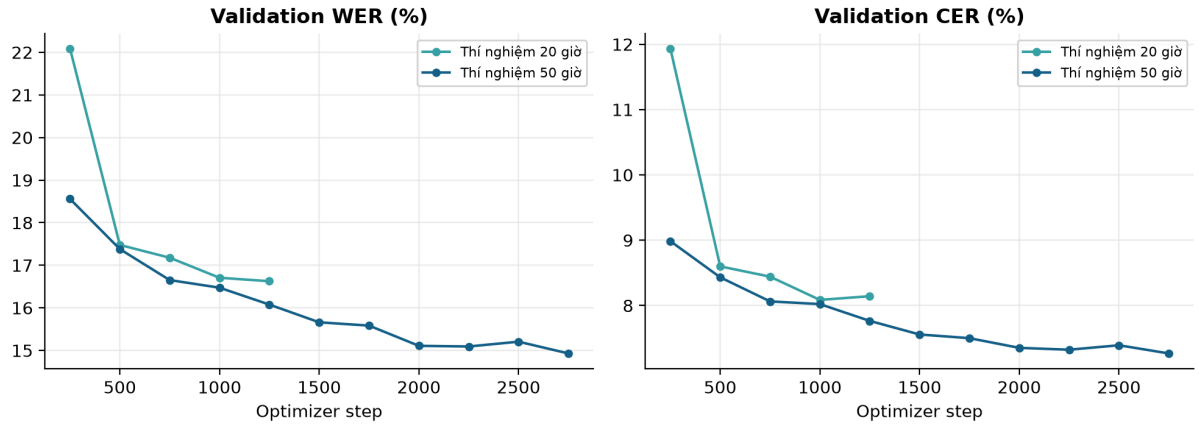
Hình 2. Kết quả trên 1.802 mẫu test; (*) chỉ là mốc tham khảo có nguy cơ chồng lấn dữ liệu.

8.1. Mức cải thiện

So sánh	Giảm WER	Giảm CER
Gốc → LoRA 20 giờ	21,56 điểm; 58,64% tương đối	12,14 điểm; 62,46%
Gốc → LoRA 50 giờ	23,00 điểm; 62,56% tương đối	12,76 điểm; 65,65%
LoRA 20 → 50 giờ	1,44 điểm; 9,47% tương đối	0,62 điểm; 8,50%

Fine-tuning mang lại cải thiện lớn so với mô hình nền. Mở rộng từ 16 lên 46 giờ train tiếp tục giảm lỗi, nhưng mức giảm nhỏ hơn so với bước chuyển từ baseline sang fine-tune. Chưa có nhiều seed hoặc khoảng tin cậy để kết luận mức ý nghĩa thống kê.

9. DIỄN BIẾN VALIDATION



Hình 3. Đường cong được dựng từ log_history trong trainer_state.json, không phải số liệu mô phỏng.

Step	20h WER	20h CER	50h WER	50h CER
250	22,09	11,93	18,56	8,99
500	17,48	8,60	17,38	8,43
750	17,18	8,44	16,65	8,06
1000	16,71	8,09	16,47	8,02
1250	16,63	8,14	16,08	7,77
1500	—	—	15,66	7,56
2000	—	—	15,11	7,35
2250	—	—	15,09	7,32
2500	—	—	15,21	7,39
2750	—	—	14,93	7,27

Mô hình 20 giờ đạt best validation WER 16,6252% tại step 1.250 và hoàn thành 1.350 steps. Mô hình 50 giờ đạt 14,9296% tại step 2.750. Đường cong giảm nhanh ở đầu và chậm dần về sau, nhưng không đủ bằng chứng để khẳng định mọi run dài hơn đều sẽ overfit.

> *Cùng một optimizer step không tương đương cùng số epoch giữa hai tập train. Ngoài kích thước dữ liệu, số lần cập nhật, thời điểm dừng và chế độ early stopping cũng tác động đến so sánh.*

10. PHÂN TÍCH EARLY STOPPING

10.1. Vì sao run 50 giờ dừng ở step 2.750?

Cấu hình cho phép tối đa 3 epoch, nhưng run dừng ở epoch 2,1351. EarlyStoppingCallback dùng patience=3 và threshold=0,2 trên metric WER được biểu diễn theo phần trăm. Ngưỡng này là 0,2 điểm phần trăm tuyệt đối, không phải yêu cầu giảm 20% tương đối.

Step	Validation WER	So với best trước đó	Bộ đếm
2.000	15,1085	Cải thiện 0,4744 điểm → đủ ngưỡng	0
2.250	15,0929	Giảm 0,0156 điểm → dưới 0,2	1
2.500	15,2057	Tăng so với best → không đạt	2
2.750	14,9296	Giảm 0,1633 điểm → dưới 0,2	3 → dừng

Step 2.750 vừa là checkpoint có WER nhỏ nhất, vừa là thời điểm dừng. Điều này không mâu thuẫn: Trainer cập nhật best metric theo giá trị tốt hơn, trong khi callback chỉ đặt lại bộ đếm khi mức cải thiện vượt ngưỡng. Ba lần liên tiếp không đủ ngưỡng dẫn đến dừng dù lần cuối vẫn tạo một cực tiểu mới.

10.2. Kết luận đúng từ dữ liệu

- Không có ba lần validation sau step 2.750 trong run đã lưu.
- Dừng sớm là quyết định của tiêu chí đã đặt, không phải chứng minh model đã đạt tối ưu toàn cục.
- WER cuối đang giảm trở lại; có thể thử ngưỡng nhỏ hơn hoặc patience lớn hơn trong một run riêng, nhưng phải chọn bằng validation.
- Không nên tiếp tục tinh chỉnh nhiều lần theo test cố định rồi coi test đó là một đánh giá cuối hoàn toàn độc lập.

Do đó, báo cáo chọn cách phát biểu thận trọng: mô hình hội tụ đủ theo quy tắc early stopping của thí nghiệm. Chưa có bằng chứng cho một kết luận tuyệt đối rằng train tiếp sẽ không cải thiện hoặc chắc chắn gây overfitting.

11. MERGE, CHUYỂN ĐỔI VÀ TRIỂN KHAI

11.1. Xuất mô hình tự huấn luyện

```
best_adapter (LoRA)
      + openai/whisper-small
      ↓ merge_and_unload()
Checkpoint Transformers hoàn chỉnh
      ↓ CTranslate2 converter, quantization=int8
Model local cho faster-whisper
      ↓ FastAPI → giao diện ViMMRC
```

merge_lora.py nạp model nền, gắn adapter bằng PEFT rồi merge_and_unload. Trọng số được lưu dưới safe serialization cùng processor. convert_ct2.py chuyển sang CTranslate2, đồng thời sao chép tokenizer.json và preprocessor_config.json để model local có đủ thành phần cần thiết.

Bản triển khai 50 giờ nằm ở models/whisper-small-vi-50h-ct2; bản 20 giờ ở models/whisper-small-vi-ct2. Mỗi thư mục đo tại thời điểm lập báo cáo khoảng 241,48 MiB, tương đương xấp xỉ 253 MB theo đơn vị thập phân. Model binary được loại khỏi Git; adapter và checkpoint huấn luyện được giữ riêng trong WSL.

11.2. Cấu hình suy luận

Thuộc tính	Giá trị production
Thư viện	faster-whisper / CTranslate2
Mặc định	CPU, compute_type=int8
Ngôn ngữ / beam	vi / beam_size=5
VAD	Bật vad_filter=True
Ngữ cảnh trước	condition_on_previous_text=False
Thứ tự chọn model	Biến môi trường override; nếu không: local 50h → local 20h → small

GPU được dùng cho train và benchmark Transformers; cấu hình server mặc định CPU INT8 để không phụ thuộc thư viện CUDA runtime trên Windows. Dung lượng nhỏ hơn và nạp được trên CPU là bằng chứng khả năng triển khai, không thay thế phép đo latency hoặc real-time factor.

12. API VÀ TÍCH HỢP ViMMRC

12.1. Hợp đồng giao tiếp

Endpoint	Chức năng
GET /	Thông tin dịch vụ và các đường dẫn sử dụng
GET /health	Trả model, device và trạng thái đã nạp model
GET /docs	Giao diện tài liệu/thử API tự động
POST /api/stt	Nhận multipart field file, trả JSON chứa text

```
POST /api/stt
Content-Type: multipart/form-data
field: file = <audio>

HTTP 200
{"text": "xin chào tôi là người việt nam"}
```

Angular gửi Blob dưới tên question.wav qua FormData. PipelineService.transcribe lấy trường text, trim rồi chuyển câu hỏi văn bản cho bước tiếp theo. Model STT được thay mà không làm đổi cấu trúc JSON của giao diện.

12.2. Quản lý tài nguyên và lỗi

- Model lazy-load qua singleton cache, tái sử dụng sau request đầu tiên.
- Inference chạy trong threadpool để không chặn trực tiếp event loop; lock tuần tự hóa truy cập model.
- Lưu audio tạm theo suffix, đóng upload và xóa file tạm trong finally.
- Giới hạn payload audio 25 MiB; kiểm tra MIME cho WAV, MP3, FLAC, OGG, WebM và một số định dạng audio khác.

Mã HTTP	Trường hợp
400 / 413 / 415	File rỗng / quá giới hạn / MIME không hỗ trợ
422	Không thu được nội dung tiếng nói
500	Lỗi nhận dạng hoặc giải mã phát sinh bên trong

Health trả model_loaded=false trước request đầu tiên là đúng thiết kế lazy-load. Health 200 không tự chứng minh model đã được nạp hoặc mọi audio đều xử lý được. Mẫu MP3 kiểm thử trả text đúng chỉ là kiểm thử chức năng, không phải tập đánh giá định lượng.

13. VẤN ĐỀ THỰC TẾ VÀ KHẢ NĂNG TÁI LẬP

Vấn đề	Cách xử lý trong quá trình thực hiện
Quy mô dataset lớn	Streaming subset; không tải đủ 131 GB; tách audio, cache, checkpoint.
Quyền truy cập Hugging Face	Chấp nhận điều kiện dataset, đăng nhập terminal; không đưa token vào repo.
Tương thích audio runtime	Đã phát hiện xung đột PyAV 19; dependency STT giới hạn av<19.
GPU auto trên Windows	Tránh phụ thuộc CUDA DLL bằng mặc định CPU INT8 cho server.
Preprocessing nhiều process	Chuyển mặc định 1 process; trích đặc trưng trước khi nạp model.
BF16 khi generation	Để Trainer quản lý autocast thay vì ép toàn bộ model sang BF16 lúc nạp train.
Arrow cache lớn / WSL I/O	Giảm writer batch; chuyển cache dataset sang ổ D, ngoài VHD trên ổ C.
Ổ C gần đầy	Dọn cache sinh lại được, kiểm tra ext4, compact VHD; xác minh lại audio trước khi train.

13.1. Bài học về lưu trữ

Dung lượng trống mà Linux báo trong ext4 không phản ánh đầy đủ dung lượng vật lý còn lại của ổ Windows chứa VHD. Cache log-Mel có thể lớn hơn đáng kể audio FLAC. Khi ổ C còn khoảng 5,2 GB, quá trình ghi cache dẫn đến lỗi I/O và root filesystem chuyển read-only. Không nên quy kết ngay là FLAC hỏng chỉ từ lỗi đọc trong trạng thái filesystem lỗi.

Sau phục hồi, kiểm tra lại toàn bộ audio và chuyển feature cache sang D giúp tiếp tục run. Trước train cần giám sát cả ổ vật lý Windows lẫn filesystem Linux, đồng thời không để Windows sleep trong phiên huấn luyện dài.

13.2. Tái lập và thời gian thực thi

Manifest, cấu hình, log_history, checkpoint và JSON metric là các bằng chứng chính. Không báo một thời gian train chính xác vì run từng bị sleep/pause và lỗi lưu trữ; wall-clock không đại diện thời gian GPU hoạt động thuần. Báo cáo cũng không suy đoán tốc độ inference khi chưa benchmark.

14. GIỚI HẠN VÀ HƯỚNG PHÁT TRIỂN

14.1. Giới hạn của thiết kế thực nghiệm

- Không speaker-disjoint: hash transcript chỉ kiểm soát trùng nội dung theo quy tắc chuẩn hóa hiện có.
- Dataset streaming subset không được chứng minh cân bằng vùng miền, giới tính, nguồn hoặc mức nhiễu.
- Chỉ có một run cho mỗi quy mô dữ liệu; chưa báo độ lệch qua nhiều seed hay khoảng tin cậy WER/CER.
- So sánh 20/50 giờ thay cả dữ liệu train, số steps và chế độ dừng; chưa cô lập hoàn toàn tác động của quy mô dữ liệu.
- ViWhisper có nguy cơ đã học dữ liệu test thuộc VietSpeech; kết quả 0,82% WER không phải mức chuẩn công bằng để kết luận mô hình tự train kém hơn bao nhiêu.
- Test cố định đã được xem nhiều lần khi phát triển; nên có thêm holdout cuối độc lập để giảm thiên lệch lựa chọn.
- Chưa đo WER/CER toàn test cho CTranslate2 INT8, chưa benchmark latency, throughput và mức tiêu thụ bộ nhớ.

14.2. Giới hạn đối với ứng dụng ViMMRC

Chưa có test set câu hỏi ViMMRC thu âm độc lập đủ lớn và chưa đo tác động STT lên chỉ số trả lời cuối của MRC. WER trung bình thấp không bảo đảm tên người, địa danh, số liệu và từ nghi vấn luôn đúng. Một lỗi ở từ khóa có thể gây tác động lớn hơn nhiều lỗi dấu câu.

14.3. Hướng phát triển ưu tiên

- Thu 100–300 câu hỏi ViMMRC đa người nói, tách người nói và nguồn với train; chấm transcript cẩn thận.
- Đánh giá đồng thời WER/CER, lỗi tên riêng/con số và chất lượng trả lời end-to-end với câu hỏi chuẩn so với câu hỏi STT.
- Benchmark lại bản INT8 trên cùng holdout, đo latency và real-time factor trên CPU/GPU.
- Phân tích lỗi trước khi tăng dữ liệu hoặc thay rank/target LoRA; dùng validation để chọn cấu hình.
- Khóa protocol chuẩn hóa transcript và chạy nhiều seed nếu tài nguyên cho phép.

15. KẾT LUẬN

Đề tài đã hoàn thành một quy trình học máy ứng dụng cho nhận dạng tiếng nói tiếng Việt, từ dữ liệu gốc đến dịch vụ sử dụng được trong ViMMRC. Việc tự fine-tune openai/whisper-small bằng LoRA tạo ra đóng góp thực hành rõ ràng hơn so với chỉ gọi mô hình tiếng Việt có sẵn.

Mốc	Kết quả chính
Dữ liệu	Streaming bộ 20 giờ; mở rộng 50 giờ, giữ validation/test cố định.
Huấn luyện	LoRA 1.769.472 tham số trainable; BF16; RTX 5060 Ti 16 GB.
Model 20 giờ	Test WER 15,20%; CER 7,29%; 3 epoch.
Model 50 giờ	Test WER 13,77%; CER 6,67%; dừng step 2.750.
So với baseline	Model 50 giờ giảm khoảng 62,56% WER tương đối.
Triển khai	Merge → CTranslate2 INT8 → faster-whisper → FastAPI → Angular.
Bảo toàn artifact	Model 20 giờ được giữ nguyên để so sánh và dự phòng.

Kết quả xác nhận fine-tuning bằng dữ liệu tiếng Việt và LoRA có thể cải thiện đáng kể Whisper Small trong điều kiện GPU cá nhân. Mở rộng dữ liệu train từ 16 lên 46 giờ tiếp tục cải thiện WER khoảng 9,47% tương đối so với mô hình 20 giờ; mức này được tính từ số đo chưa làm tròn.

Tuy nhiên, phạm vi kết luận giới hạn ở test subset VietSpeech và protocol hiện tại. Chưa thể khẳng định mô hình đạt một ngưỡng chất lượng phổ quát cho tiếng Việt, cũng chưa chứng minh hiệu quả cuối trên mọi câu hỏi ViMMRC. Cần kiểm thử độc lập đúng miền và đánh giá lại bản production trước khi đưa ra tuyên bố mạnh hơn.

Giá trị đối với học phần

Quá trình thực hiện thể hiện các nội dung trọng tâm của môn Học máy tiên tiến và ứng dụng: học chuyển giao, PEFT/LoRA, mixed precision, thiết kế tập dữ liệu, chống rò rỉ dữ liệu, metric chuỗi, early stopping, quản lý thí nghiệm và chuyển mô hình từ môi trường nghiên cứu sang ứng dụng.

Các thông tin trường, lớp, giảng viên và thành viên trên bìa để trống có chủ đích; cần điền theo thông tin chính thức trước khi nộp.

TÀI LIỆU THAM KHẢO

[1] Radford, A. và cộng sự. Robust Speech Recognition via Large-Scale Weak Supervision. 2022/2023.
https://arxiv.org/abs/2212.04356

[2] OpenAI / Hugging Face. Model card: openai/whisper-small.
https://huggingface.co/openai/whisper-small

[3] Hu, E. J. và cộng sự. LoRA: Low-Rank Adaptation of Large Language Models. 2021/2022.
https://arxiv.org/abs/2106.09685

[4] Pham Quang Nhut, Duong Pham Hoang Anh, Nguyen Vinh Tiep. VietSpeech: Vietnamese social voice dataset. 2024.
https://huggingface.co/datasets/NhutP/VietSpeech

[5] NhutP. Model card: ViWhisper-small.
https://huggingface.co/NhutP/ViWhisper-small

[6] Hugging Face. PEFT và Transformers: tài liệu LoRA và Seq2SeqTrainer.
https://huggingface.co/docs/peft/conceptual_guides/lora

[7] SYSTRAN. faster-whisper: Whisper inference with CTranslate2.
https://github.com/SYSTRAN/faster-whisper

[8] OpenNMT. CTranslate2: Transformers conversion và quantization.
https://opennmt.net/CTranslate2/guides/transformers.html

[9] JiWER. Thư viện tính WER/CER.
https://jitsi.github.io/jiwer/

Nguồn minh chứng nội bộ

Số liệu chính được lấy từ speech-to-text_Luong/training/results_20h.json và results_50h.json, đối chiếu trực tiếp với trainer_state.json của checkpoint-1350 (run 20 giờ) và checkpoint-2750 (run 50 giờ). Mã nguồn chuẩn bị dữ liệu, train, evaluate, merge, convert và server được đọc để mô tả đúng hành vi đã triển khai.

Ngày truy cập/đối chiếu: 25/10/2026. Tài liệu ngoài được dùng để mô tả phương pháp; các số đo trong báo cáo là kết quả thí nghiệm nội bộ, không phải kết quả tác giả Whisper hoặc VietSpeech công bố.

PHỤ LỤC A. LỆNH TÁI LẬP THÍ NGHIỆM

Các lệnh dưới đây chạy trong Ubuntu WSL, từ thư mục gốc repository. Dùng thư mục output mới cho run tái lập; không ghi đè dữ liệu hoặc model đã có. Chuẩn bị dữ liệu cần quyền truy cập VietSpeech và kết nối mạng.

```
source ~/venvs/whisper-train/bin/activate
# cd đến repository trong WSL trước khi chạy các script
```

A.1. Dữ liệu 20 giờ và mở rộng 50 giờ

```
~/venvs/vietspeech-data/bin/python \
  speech-to-text_Luong/training/prepare_vietspeech.py \
  --hours 20 --output-dir ~/vimmrc-training/data/vietspeech_20h

~/venvs/vietspeech-data/bin/python \
  speech-to-text_Luong/training/expand_vietspeech.py \
  --base-dir ~/vimmrc-training/data/vietspeech_20h \
  --output-dir ~/vimmrc-training/data/vietspeech_50h \
  --total-hours 50

~/venvs/vietspeech-data/bin/python \
  speech-to-text_Luong/training/validate_expansion.py \
  --base-dir ~/vimmrc-training/data/vietspeech_20h \
  --expanded-dir ~/vimmrc-training/data/vietspeech_50h
```

A.2. Fine-tune 50 giờ

```
python speech-to-text_Luong/training/train_lora.py \
  --data-dir ~/vimmrc-training/data/vietspeech_50h \
  --output-dir ~/vimmrc-training/checkpoints/whisper-small-vi-50h-lora \
  --cache-dir /mnt/d/vimmrc-training-cache/datasets \
  --epochs 3 --batch-size 4 --gradient-accumulation 8 \
  --learning-rate 1e-4 --eval-steps 250 --save-steps 250 \
  --early-stopping-patience 3 --early-stopping-threshold 0.2
```

A.3. Theo dõi và yêu cầu lưu trữ

Theo dõi bằng TensorBoard hoặc log terminal; kiểm tra GPU bằng nvidia-smi. Cache feature đặt trên ổ D để tránh đầy VHD chứa Ubuntu trên C. Giữ máy hoạt động trong phiên train dài và giám sát dung lượng ổ vật lý. Bộ dữ liệu 50 giờ hiện có không cần tạo lại nếu manifest/audio đã được xác minh.

Các đường dẫn Linux trong phụ lục là đường dẫn môi trường WSL của thí nghiệm; không chạy nguyên lệnh Bash trực tiếp trong PowerShell.

PHỤ LỤC B. ĐÁNH GIÁ, TRIỂN KHAI VÀ ARTIFACT

B.1. Merge, đánh giá, chuyển đổi

```
python speech-to-text_Luong/training/merge_lora.py \
  --adapter ~/vimmrc-training/checkpoints/whisper-small-vi-50h-lora/best_adapter \
  --output-dir ~/vimmrc-training/models/whisper-small-vi-50h-merged

python speech-to-text_Luong/training/evaluate_models.py \
  --manifest ~/vimmrc-training/data/vietspeech_50h/test.jsonl \
  --models ~/vimmrc-training/models/whisper-small-vi-merged \
           ~/vimmrc-training/models/whisper-small-vi-50h-merged \
  --output ~/vimmrc-training/reports/stt_20h_vs_50h.json

python speech-to-text_Luong/training/convert_ct2.py \
  --model ~/vimmrc-training/models/whisper-small-vi-50h-merged \
  --output-dir ~/vimmrc-training/models/whisper-small-vi-50h-ct2
```

B.2. Thử API trên Windows

Sau khi chép model CTranslate2 vào thư mục models của STT, chạy từ thư mục speech-to-text_Luong bằng môi trường server đã cài dependency:

```
.\.venv\Scripts\python.exe -m uvicorn stt_server:app --port 8001

curl.exe http://localhost:8001/health
curl.exe -X POST http://localhost:8001/api/stt `
  -F "file=@whisper_vi_test.mp3;type=audio/mpeg"
```

B.3. Tệp minh chứng trong repository

Vị trí tương đối từ speech-to-text_Luong	Vai trò
training/results_20h.json	Thông số và metric thí nghiệm 20 giờ
training/results_50h.json	Thông số và metric thí nghiệm 50 giờ
training/train_lora.py	Huấn luyện LoRA và chọn best adapter
training/evaluate_models.py	Đánh giá chung WER/CER
training/README.md	Hướng dẫn pipeline và lệnh chạy
stt_server.py	Dịch vụ inference production
models/whisper-small-vi-50h-ct2	Trọng số INT8 local, không commit Git